

✓ Fruit Classification ANN — ReLU vs Leaky ReLU

This notebook contains an experiment comparing two activation functions, **ReLU** and **Leaky ReLU**, on an *Artificial Neural Network* (ANN) architecture for fruit image classification using the **Fruit360** dataset.

1. Install Library

Before running the notebook, make sure all required libraries are installed.

Library	Purpose
<code>opencv-python</code>	Reading and processing images
<code>tqdm</code>	Progress bar during dataset loading
<code>scikit-image</code>	Additional image operations
<code>kagglehub</code>	Download datasets directly from Kaggle
<code>streamlit</code>	Building interactive web applications

```
!pip install -q opencv-python tqdm scikit-image kagglehub streamlit
print('All packages installed!')
```

9.2/9.2 MB 22.0 MB/s eta 0:00:00

11.3/11.3 MB 44.2 MB/s eta 0:00:00

All packages installed!

✓ 2. Import Library

Importing all libraries that will be used throughout the notebook.

- **TensorFlow / Keras** → building and training the ANN model
- **NumPy** → array and matrix operations
- **Pandas** → tabular data manipulation
- **Matplotlib** → chart and graph visualization
- **OpenCV (cv2)** → image preprocessing
- **os / json** → file management and configuration storage
- **tqdm** → progress bar during dataset iteration
- **kagglehub** → downloading datasets from Kaggle

`np.random.seed(42)` and `tf.random.set_seed(42)` are set to ensure the experiment results are reproducible.

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import cv2
import json

from tqdm import tqdm

from tensorflow.keras.models import (
    Sequential,
    load_model
)

from tensorflow.keras.layers import (
    Dense,
    ReLU,
    LeakyReLU
)

from tensorflow.keras.utils import (
    to_categorical
)

import kagglehub

np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow Version:", tf.__version__)
```

TensorFlow Version: 2.20.0

3. Download Dataset

The dataset used is **Fruit360** from Kaggle (`moltean/fruits`).

This dataset contains fruit images sized **100×100 pixels** with a white background, organized into *Training* and *Test* folders named after each fruit class.

`kagglehub.dataset_download()` will automatically download the dataset to the local cache directory.

```
print("Downloading dataset...")

path = kagglehub.dataset_download(
    "moltean/fruits"
)

print("Dataset downloaded to:")
print(path)
```

```
Downloading dataset...
Using Colab cache for faster access to the 'fruits' dataset.
Dataset downloaded to:
/kaggle/input/fruits
```

4. Dataset Configuration

Defining the dataset configuration before the loading process:

- `train_path` and `test_path` → paths to the Training and Test folders inside the Fruit360 dataset
- `classes` → 10 fruit classes selected for this experiment
- `IMG_SIZE = 64` → all images will be resized to 64×64 pixels for memory efficiency and faster training
- `INPUT_DIM = 64 × 64 × 3 = 12,288` → total input neurons (the length of the vector after the image is *flattened*)

After the configuration is defined, the class list is saved to `classes.json` so it can be read by the Streamlit application.

```
train_path = os.path.join(
    path,
    "fruits-360_100x100",
    "fruits-360",
    "Training"
)

test_path = os.path.join(
    path,
    "fruits-360_100x100",
    "fruits-360",
    "Test"
)

classes = [
    "Apple Red 1",
    "Banana 1",
    "Raspberry 3",
    "Avocado 1",
    "Kiwi 1",
    "Carrot 1",
    "Papaya 1",
    "Strawberry 1",
    "Pear 10",
    "Tomato 1"
]

IMG_SIZE = 64

INPUT_DIM = IMG_SIZE * IMG_SIZE * 3

print("Training Path :", train_path)
print("Test Path      :", test_path)
print("Classes        :", len(classes))
print("Input Dim       :", INPUT_DIM)
```

```
Training Path : /kaggle/input/fruits/fruits-360_100x100/fruits-360/Training
Test Path      : /kaggle/input/fruits/fruits-360_100x100/fruits-360/Test
Classes        : 10
Input Dim      : 12288
```

```
# Save the class list to a JSON file
# This file will be read by the Streamlit app
# to ensure prediction label order is consistent with the training order
with open(
```

```

        "classes.json",
        "w"
    ) as f:

        json.dump(
            classes,
            f
        )

    print("Classes saved!")

```

Classes saved!

5. Preprocessing Function

The `preprocess_image()` function converts a raw image into a numerical vector ready to be fed into the model.

Preprocessing steps:

1. **Read image** → if the input is a file path, the image is read using `cv2.imread()` (in BGR format)
2. **Convert BGR → RGB** → OpenCV reads images in BGR format, while the model is trained with RGB, so conversion is necessary
3. **Resize to 64×64** → standardizes all image sizes so the input dimension is consistent
4. **Normalize (÷ 255)** → converts pixel values from the range [0, 255] to [0.0, 1.0] for more stable training and faster convergence
5. **Flatten** → converts the 2D array (64×64×3) into a 1D vector of length 12,288, since Dense layers require a vector as input

```

def preprocess_image(
    img_input,
    is_path=False
):

    if is_path:

        img = cv2.imread(
            img_input,
            cv2.IMREAD_COLOR
        )

        if img is None:

            raise ValueError(
                f"Cannot read image: {img_input}"
            )

    else:

        img = img_input.copy()

    # Convert BGR (OpenCV format) to RGB
    img = cv2.cvtColor(
        img,
        cv2.COLOR_BGR2RGB
    )

    # Resize to target dimensions
    img = cv2.resize(
        img,
        (IMG_SIZE, IMG_SIZE)
    )

    # Normalize pixel values to the range [0, 1]
    img = img.astype(
        "float32"
    ) / 255.0

    # Flatten into a 1D vector
    img = img.flatten()

    return img

```

6. Load Dataset Function

The `load_dataset()` function reads all images from the dataset folder and converts them into NumPy arrays.

How it works:

- Iterates through each fruit class (10 classes)
- For each class, reads all image files inside its folder
- Each image is processed with `preprocess_image()` and the result is appended to list `X`

- A numeric label (0–9) is appended to list `Y` based on the class index

If any image fails to load (corrupted or unsupported format), it is skipped without stopping the loading process.

The final result is returned as a tuple `(X, Y)` in NumPy array format.

```
def load_dataset(folder_root):

    X = []
    Y = []

    for label, fruit in enumerate(classes):

        folder_path = os.path.join(
            folder_root,
            fruit
        )

        print(f"\nLoading {fruit}")

        for img_name in tqdm(
            os.listdir(folder_path)
        ):

            img_path = os.path.join(
                folder_path,
                img_name
            )

            try:

                img_flat = preprocess_image(
                    img_path,
                    is_path=True
                )

                X.append(img_flat)

                Y.append(label)

            except Exception as e:

                print(
                    f"Skip {img_name}: {e}"
                )

    return (
        np.array(X, dtype="float32"),
        np.array(Y, dtype="int32")
    )
```

7. Load Training & Test Dataset

Calls `load_dataset()` to load the training and test data into memory.

- **x_train / x_test** → flattened image arrays, shape: `(N, 12288)`
- **y_train / y_test** → numeric labels for each image, shape: `(N,)`

This process may take several minutes depending on the dataset size and machine specifications.

```
# Load training dataset
print("Loading Training Dataset...")

x_train, y_train = load_dataset(
    train_path
)

# Load test dataset
print("\nLoading Test Dataset...")

x_test, y_test = load_dataset(
    test_path
)
```

```
Loading Banana 1
100%|██████████| 490/490 [00:02<00:00, 185.53it/s]
```

```

Loading Kiwi 1
100%|██████████| 466/466 [00:02<00:00, 164.54it/s]

Loading Carrot 1
100%|██████████| 151/151 [00:00<00:00, 195.56it/s]

Loading Papaya 1
100%|██████████| 492/492 [00:02<00:00, 190.22it/s]

Loading Strawberry 1
100%|██████████| 492/492 [00:02<00:00, 184.80it/s]

Loading Pear 10
100%|██████████| 701/701 [00:03<00:00, 188.13it/s]

Loading Tomato 1
100%|██████████| 738/738 [00:04<00:00, 177.38it/s]

Loading Test Dataset...

Loading Apple Red 1
100%|██████████| 164/164 [00:00<00:00, 171.34it/s]

Loading Banana 1
100%|██████████| 166/166 [00:01<00:00, 162.29it/s]

Loading Raspberry 3
100%|██████████| 147/147 [00:00<00:00, 161.95it/s]

Loading Avocado 1
100%|██████████| 143/143 [00:00<00:00, 154.33it/s]

Loading Kiwi 1
100%|██████████| 156/156 [00:01<00:00, 150.11it/s]

Loading Carrot 1
100%|██████████| 50/50 [00:00<00:00, 171.36it/s]

Loading Papaya 1
100%|██████████| 164/164 [00:01<00:00, 149.26it/s]

Loading Strawberry 1
100%|██████████| 164/164 [00:01<00:00, 153.92it/s]

Loading Pear 10
100%|██████████| 231/231 [00:01<00:00, 163.90it/s]

Loading Tomato 1
100%|██████████| 246/246 [00:01<00:00, 155.56it/s]

```

```

# Display dataset shape information
print("\nTrain Shape :", x_train.shape)
print("Test Shape  :", x_test.shape)

```

```

Train Shape : (4891, 12288)
Test Shape  : (1631, 12288)

```

8. One Hot Encoding

The labels in `y_train` and `y_test` are currently integer values (0–9). A model with a *Softmax* output layer requires labels in **One Hot Encoding** format.

Conversion example:

- Label 3 (Avocado) → `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`
- Label 4 (Kiwi) → `[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]`

`to_categorical()` from Keras performs this conversion automatically. After conversion, the label shape changes from `(N,)` to `(N, 10)`.

```

y_train = to_categorical(
    y_train,
    num_classes=len(classes)
)

y_test = to_categorical(
    y_test,
    num_classes=len(classes)
)

print(
    "Train Label Shape:",
    y_train.shape
)

```

```
print(
    "Test Label Shape:",
    y_test.shape
)
```

```
Train Label Shape: (4891, 10)
Test Label Shape: (1631, 10)
```

9. Build & Train ReLU Model

Model Architecture

The first model uses **ReLU** (*Rectified Linear Unit*) as its activation function.

```
Input   : 12,288 neurons (64×64×3 pixels, flattened)
Dense   : 256 neurons → ReLU
Dense   : 128 neurons → ReLU
Dense   : 64  neurons → ReLU
Output  : 10  neurons → Softmax (one per fruit class)
```

Why ReLU? ReLU returns 0 for negative inputs and the original value for positive inputs: $f(x) = \max(0, x)$. It is computationally efficient, helps address the *vanishing gradient* problem, and generally performs well in deep networks.

```
# Build the ReLU model architecture
relu_model = Sequential([

    Dense(
        256,
        input_shape=(INPUT_DIM,)
    ),

    ReLU(),

    Dense(128),

    ReLU(),

    Dense(64),

    ReLU(),

    Dense(
        len(classes),
        activation="softmax"
    )

], name="ReLU_Model")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Compile ReLU Model

Before training, the model must be *compiled* by specifying:

- **Optimizer** `adam` → an adaptive optimization algorithm that combines *momentum* and *RMSProp*. Adam is widely used because it converges faster than standard SGD
- **Loss** `categorical_crossentropy` → the standard loss function for multi-class classification with One Hot Encoded labels
- **Metrics** `accuracy` → the evaluation metric displayed during training

```
# Compile the ReLU model
relu_model.compile(

    optimizer="adam",

    loss="categorical_crossentropy",

    metrics=["accuracy"]

)

relu_model.summary()
```

Model: "ReLU_Model"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	3,145,984
re_lu (ReLU)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
re_lu_1 (ReLU)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
re_lu_2 (ReLU)	(None, 64)	0
dense_3 (Dense)	(None, 10)	650

Total params: 3,187,786 (12.16 MB)

Trainable params: 3,187,786 (12.16 MB)

Train ReLU Model

The model is trained for 30 epochs with a batch size of 32.

- epoch → one complete pass through the entire training dataset
- batch_size → the number of samples processed before the model updates its weights
- validation_data → the test data is used to monitor model performance at each epoch, enabling detection of *overfitting*

```
# Train the ReLU model
history_relu = relu_model.fit(

    x_train,
    y_train,

    validation_data=(
        x_test,
        y_test
    ),

    epochs=30,

    batch_size=32,

    verbose=1
)
```

```
Epoch 2/30
153/153 ————— 9s 59ms/step - accuracy: 0.9845 - loss: 0.0506 - val_accuracy: 0.9828 - val_loss: 0.0438
Epoch 3/30
153/153 ————— 8s 51ms/step - accuracy: 1.0000 - loss: 0.0035 - val_accuracy: 0.9871 - val_loss: 0.0322
Epoch 4/30
153/153 ————— 7s 45ms/step - accuracy: 1.0000 - loss: 0.0019 - val_accuracy: 0.9908 - val_loss: 0.0281
Epoch 5/30
153/153 ————— 8s 51ms/step - accuracy: 1.0000 - loss: 0.0010 - val_accuracy: 0.9908 - val_loss: 0.0251
Epoch 6/30
153/153 ————— 7s 47ms/step - accuracy: 1.0000 - loss: 7.0226e-04 - val_accuracy: 0.9902 - val_loss: 0.0251
Epoch 7/30
153/153 ————— 8s 52ms/step - accuracy: 1.0000 - loss: 4.8619e-04 - val_accuracy: 0.9902 - val_loss: 0.0249
Epoch 8/30
153/153 ————— 9s 57ms/step - accuracy: 1.0000 - loss: 3.4052e-04 - val_accuracy: 0.9902 - val_loss: 0.0235
Epoch 9/30
153/153 ————— 8s 51ms/step - accuracy: 1.0000 - loss: 2.6228e-04 - val_accuracy: 0.9902 - val_loss: 0.0230
Epoch 10/30
153/153 ————— 9s 58ms/step - accuracy: 1.0000 - loss: 2.0800e-04 - val_accuracy: 0.9902 - val_loss: 0.0221
Epoch 11/30
153/153 ————— 7s 45ms/step - accuracy: 1.0000 - loss: 1.6884e-04 - val_accuracy: 0.9902 - val_loss: 0.0217
Epoch 12/30
153/153 ————— 8s 55ms/step - accuracy: 1.0000 - loss: 1.3900e-04 - val_accuracy: 0.9902 - val_loss: 0.0212
Epoch 13/30
153/153 ————— 7s 48ms/step - accuracy: 1.0000 - loss: 1.1590e-04 - val_accuracy: 0.9902 - val_loss: 0.0209
Epoch 14/30
153/153 ————— 9s 56ms/step - accuracy: 1.0000 - loss: 9.7702e-05 - val_accuracy: 0.9902 - val_loss: 0.0206
Epoch 15/30
153/153 ————— 10s 58ms/step - accuracy: 1.0000 - loss: 8.2967e-05 - val_accuracy: 0.9908 - val_loss: 0.0201
Epoch 16/30
153/153 ————— 10s 57ms/step - accuracy: 1.0000 - loss: 7.0829e-05 - val_accuracy: 0.9908 - val_loss: 0.0196
Epoch 17/30
153/153 ————— 7s 49ms/step - accuracy: 1.0000 - loss: 6.0637e-05 - val_accuracy: 0.9908 - val_loss: 0.0191
Epoch 18/30
153/153 ————— 8s 54ms/step - accuracy: 1.0000 - loss: 5.2369e-05 - val_accuracy: 0.9908 - val_loss: 0.0193
Epoch 19/30
153/153 ————— 11s 59ms/step - accuracy: 1.0000 - loss: 4.5451e-05 - val_accuracy: 0.9908 - val_loss: 0.0184
Epoch 20/30
153/153 ————— 8s 49ms/step - accuracy: 1.0000 - loss: 3.9407e-05 - val_accuracy: 0.9920 - val_loss: 0.0171
Epoch 21/30
```

```

153/153 — 9s 58ms/step - accuracy: 1.0000 - loss: 3.0158e-05 - val_accuracy: 0.9920 - val_loss: 0.0169
Epoch 23/30
153/153 — 7s 46ms/step - accuracy: 1.0000 - loss: 2.6460e-05 - val_accuracy: 0.9920 - val_loss: 0.0166
Epoch 24/30
153/153 — 8s 53ms/step - accuracy: 1.0000 - loss: 2.3206e-05 - val_accuracy: 0.9920 - val_loss: 0.0160
Epoch 25/30
153/153 — 7s 48ms/step - accuracy: 1.0000 - loss: 2.0263e-05 - val_accuracy: 0.9920 - val_loss: 0.0158
Epoch 26/30
153/153 — 9s 57ms/step - accuracy: 1.0000 - loss: 1.7601e-05 - val_accuracy: 0.9920 - val_loss: 0.0151
Epoch 27/30
153/153 — 8s 53ms/step - accuracy: 1.0000 - loss: 1.5254e-05 - val_accuracy: 0.9926 - val_loss: 0.0146
Epoch 28/30
153/153 — 8s 49ms/step - accuracy: 1.0000 - loss: 1.3235e-05 - val_accuracy: 0.9926 - val_loss: 0.0140
Epoch 29/30
153/153 — 11s 57ms/step - accuracy: 1.0000 - loss: 1.1461e-05 - val_accuracy: 0.9926 - val_loss: 0.0138
Epoch 30/30
153/153 — 8s 55ms/step - accuracy: 1.0000 - loss: 9.9370e-06 - val_accuracy: 0.9926 - val_loss: 0.0138

```

Evaluate ReLU Model

After training is complete, the model is evaluated on the test data to obtain the final **accuracy** and **loss** values.

```

# Evaluate the ReLU model on the test data
relu_loss, relu_acc = relu_model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(
    "ReLU Accuracy:",
    relu_acc
)

```

```
ReLU Accuracy: 0.9926425218582153
```

```

# Save the full ReLU model (architecture + weights)
relu_model.save(
    "fruit_relu_model.keras"
)

print(
    "ReLU model saved!"
)

```

```
ReLU model saved!
```

10. Build & Train Leaky ReLU Model

Model Architecture

The second model uses **Leaky ReLU** as its activation function with `negative_slope = 0.1`.

```

Input   : 12,288 neurons
Dense   : 256 neurons → Leaky ReLU (α=0.1)
Dense   : 128 neurons → Leaky ReLU (α=0.1)
Dense   : 64  neurons → Leaky ReLU (α=0.1)
Output  : 10  neurons → Softmax

```

Difference from ReLU: ReLU outputs 0 for all negative inputs, which can cause neurons to "die" (*dying ReLU problem*) if weights consistently produce negative values. Leaky ReLU addresses this by allowing a small gradient for negative values: $f(x) = x$ if $x > 0$, and $f(x) = \alpha \cdot x$ if $x \leq 0$. With $\alpha = 0.1$, negative inputs are still passed through at 10% of their original scale.

```

# Build the Leaky ReLU model architecture
leaky_model = Sequential([

    Dense(
        256,
        input_shape=(INPUT_DIM,)
    ),

    LeakyReLU(
        negative_slope=0.1
    ),

    Dense(128),

    LeakyReLU(
        negative_slope=0.1
    ),

```



```

    ),

    Dense(64),

    LeakyReLU(
        negative_slope=0.1
    ),

    Dense(
        len(classes),
        activation="softmax"
    )

], name="LeakyReLU_Model")

```

```

# Compile the Leaky ReLU model
# Same configuration as ReLU to ensure a fair comparison
leaky_model.compile(

    optimizer="adam",

    loss="categorical_crossentropy",

    metrics=["accuracy"]

)

leaky_model.summary()

```

Model: "LeakyReLU_Model"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 256)	3,145,984
leaky_re_lu (LeakyReLU)	(None, 256)	0
dense_5 (Dense)	(None, 128)	32,896
leaky_re_lu_1 (LeakyReLU)	(None, 128)	0
dense_6 (Dense)	(None, 64)	8,256
leaky_re_lu_2 (LeakyReLU)	(None, 64)	0
dense_7 (Dense)	(None, 10)	650

Total params: 3,187,786 (12.16 MB)

Trainable params: 3,187,786 (12.16 MB)

```

# Train the Leaky ReLU model
# Same epochs and batch size as ReLU for a valid comparison
history_leaky = leaky_model.fit(

    x_train,
    y_train,

    validation_data=(
        x_test,
        y_test
    ),

    epochs=30,

    batch_size=32,

    verbose=1

)

```

```

Epoch 2/30
153/153 ————— 7s 47ms/step - accuracy: 0.9806 - loss: 0.0606 - val_accuracy: 0.9828 - val_loss: 0.0526
Epoch 3/30
153/153 ————— 8s 54ms/step - accuracy: 0.9996 - loss: 0.0037 - val_accuracy: 0.9841 - val_loss: 0.0494
Epoch 4/30
153/153 ————— 7s 44ms/step - accuracy: 1.0000 - loss: 0.0017 - val_accuracy: 0.9847 - val_loss: 0.0448
Epoch 5/30
153/153 ————— 8s 54ms/step - accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 0.9859 - val_loss: 0.0447
Epoch 6/30
153/153 ————— 7s 45ms/step - accuracy: 1.0000 - loss: 6.5894e-04 - val_accuracy: 0.9865 - val_loss: 0.0445
Epoch 7/30
153/153 ————— 9s 56ms/step - accuracy: 1.0000 - loss: 4.7366e-04 - val_accuracy: 0.9865 - val_loss: 0.0420
Epoch 8/30
153/153 ————— 8s 54ms/step - accuracy: 1.0000 - loss: 3.5104e-04 - val_accuracy: 0.9865 - val_loss: 0.0404
Epoch 9/30
153/153 ————— 9s 49ms/step - accuracy: 1.0000 - loss: 2.6938e-04 - val_accuracy: 0.9865 - val_loss: 0.0394
Epoch 10/30

```

```

153/153 — 9s 58ms/step - accuracy: 1.0000 - loss: 1.5950e-04 - val_accuracy: 0.9865 - val_loss: 0.0370
Epoch 12/30
153/153 — 8s 49ms/step - accuracy: 1.0000 - loss: 1.2835e-04 - val_accuracy: 0.9865 - val_loss: 0.0358
Epoch 13/30
153/153 — 12s 58ms/step - accuracy: 1.0000 - loss: 1.0412e-04 - val_accuracy: 0.9865 - val_loss: 0.0346
Epoch 14/30
153/153 — 9s 60ms/step - accuracy: 1.0000 - loss: 8.5127e-05 - val_accuracy: 0.9865 - val_loss: 0.0348
Epoch 15/30
153/153 — 8s 47ms/step - accuracy: 1.0000 - loss: 6.9666e-05 - val_accuracy: 0.9865 - val_loss: 0.0349
Epoch 16/30
153/153 — 10s 45ms/step - accuracy: 1.0000 - loss: 5.8455e-05 - val_accuracy: 0.9865 - val_loss: 0.0350
Epoch 17/30
153/153 — 12s 55ms/step - accuracy: 1.0000 - loss: 4.9648e-05 - val_accuracy: 0.9865 - val_loss: 0.0347
Epoch 18/30
153/153 — 11s 62ms/step - accuracy: 1.0000 - loss: 4.2808e-05 - val_accuracy: 0.9865 - val_loss: 0.0341
Epoch 19/30
153/153 — 8s 55ms/step - accuracy: 1.0000 - loss: 3.7315e-05 - val_accuracy: 0.9865 - val_loss: 0.0336
Epoch 20/30
153/153 — 9s 56ms/step - accuracy: 1.0000 - loss: 3.2316e-05 - val_accuracy: 0.9865 - val_loss: 0.0337
Epoch 21/30
153/153 — 10s 58ms/step - accuracy: 1.0000 - loss: 2.8377e-05 - val_accuracy: 0.9871 - val_loss: 0.0336
Epoch 22/30
153/153 — 8s 53ms/step - accuracy: 1.0000 - loss: 2.5150e-05 - val_accuracy: 0.9877 - val_loss: 0.0331
Epoch 23/30
153/153 — 9s 61ms/step - accuracy: 1.0000 - loss: 2.2259e-05 - val_accuracy: 0.9877 - val_loss: 0.0327
Epoch 24/30
153/153 — 9s 59ms/step - accuracy: 1.0000 - loss: 1.9753e-05 - val_accuracy: 0.9877 - val_loss: 0.0327
Epoch 25/30
153/153 — 10s 59ms/step - accuracy: 1.0000 - loss: 1.7486e-05 - val_accuracy: 0.9877 - val_loss: 0.0326
Epoch 26/30
153/153 — 8s 53ms/step - accuracy: 1.0000 - loss: 1.5519e-05 - val_accuracy: 0.9871 - val_loss: 0.0322
Epoch 27/30
153/153 — 9s 58ms/step - accuracy: 1.0000 - loss: 1.3846e-05 - val_accuracy: 0.9871 - val_loss: 0.0317
Epoch 28/30
153/153 — 7s 47ms/step - accuracy: 1.0000 - loss: 1.2430e-05 - val_accuracy: 0.9871 - val_loss: 0.0312
Epoch 29/30
153/153 — 8s 55ms/step - accuracy: 1.0000 - loss: 1.1194e-05 - val_accuracy: 0.9871 - val_loss: 0.0311
Epoch 30/30
153/153 — 9s 56ms/step - accuracy: 1.0000 - loss: 1.0100e-05 - val_accuracy: 0.9871 - val_loss: 0.0306

```

```

# Evaluate the Leaky ReLU model on the test data
leaky_loss, leaky_acc = leaky_model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(
    "Leaky ReLU Accuracy:",
    leaky_acc
)

```

```
Leaky ReLU Accuracy: 0.9871244430541992
```

11. Model Comparison

Comparing the performance of both models based on **accuracy** and **loss** on the test data.

Winner determination logic:

1. The model with **higher accuracy** wins directly
2. If accuracy is exactly equal, the model with **lower loss** wins (lower loss = more confident and precise predictions)

```

print("="*50)

print(
    f"{'Model':<15}"
    f"{'Accuracy':>12}"
    f"{'Loss':>12}"
)

print("="*50)

print(
    f"{'ReLU':<15}"
    f"{'relu_acc':>12.4f}"
    f"{'relu_loss':>12.6f}"
)

print(
    f"{'Leaky ReLU':<15}"
    f"{'leaky_acc':>12.4f}"
    f"{'leaky_loss':>12.6f}"
)

print("="*50)

```

```
# Determine the winner based on accuracy, with loss as a tiebreaker
if relu_acc > leaky_acc:

    winner = "ReLU"

elif leaky_acc > relu_acc:

    winner = "Leaky ReLU"

else:

    winner = (
        "ReLU"
        if relu_loss < leaky_loss
        else "Leaky ReLU"
    )

print("Winner :", winner)
```

```
=====
Model            Accuracy      Loss
=====
ReLU             0.9926      0.013818
Leaky ReLU       0.9871      0.030569
=====
Winner : ReLU
```

12. Accuracy & Loss Curve

Visualizing the training and validation curves for both models over 30 epochs.

These curves are important for detecting:

- **Underfitting** → both training and validation accuracy are consistently low
- **Overfitting** → training accuracy is high but validation accuracy is significantly lower (the model has memorized the training data)
- **Normal convergence** → both curves approach a stable value together

Four graphs are displayed: Accuracy and Loss for each model (ReLU & Leaky ReLU).

```
fig, axes = plt.subplots(
    2,
    2,
    figsize=(14,10)
)

# ReLU Accuracy
axes[0,0].plot(
    history_relu.history['accuracy'],
    label='Train'
)

axes[0,0].plot(
    history_relu.history['val_accuracy'],
    label='Validation'
)

axes[0,0].set_title(
    'ReLU Accuracy'
)

axes[0,0].legend()

# ReLU Loss
axes[0,1].plot(
    history_relu.history['loss'],
    label='Train'
)

axes[0,1].plot(
    history_relu.history['val_loss'],
    label='Validation'
)

axes[0,1].set_title(
    'ReLU Loss'
)

axes[0,1].legend()

# Leaky ReLU Accuracy
axes[1,0].plot(
    history_leaky.history['accuracy'],
```

```
        label='Train'
    )

    axes[1,0].plot(
        history_leaky.history['val_accuracy'],
        label='Validation'
    )

    axes[1,0].set_title(
        'Leaky ReLU Accuracy'
    )

    axes[1,0].legend()

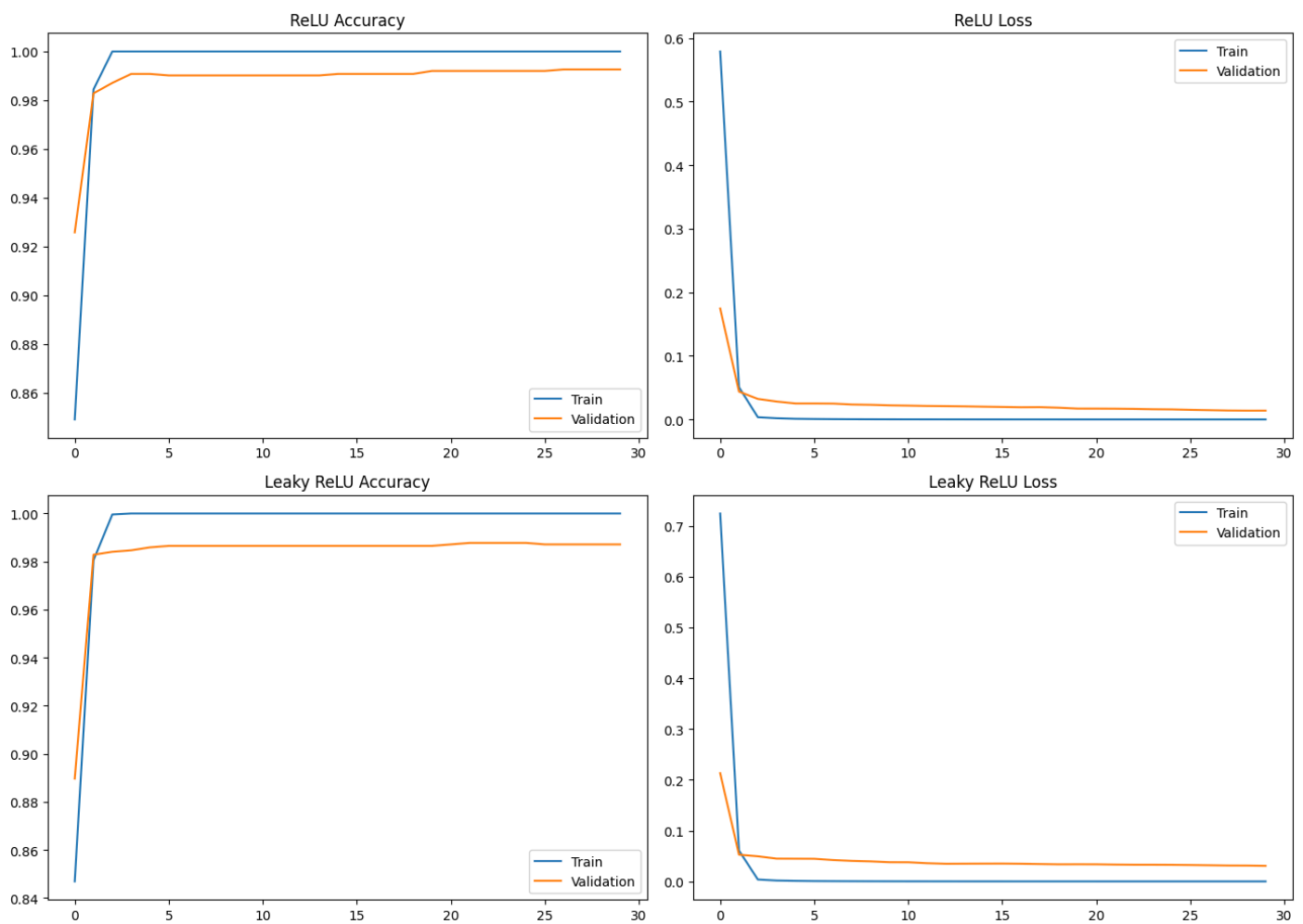
# Leaky ReLU Loss
axes[1,1].plot(
    history_leaky.history['loss'],
    label='Train'
)

axes[1,1].plot(
    history_leaky.history['val_loss'],
    label='Validation'
)

axes[1,1].set_title(
    'Leaky ReLU Loss'
)

axes[1,1].legend()

plt.tight_layout()
plt.show()
```



13. Confusion Matrix — ReLU

A **Confusion Matrix** displays the detailed prediction results of a model in an $N \times N$ table (N = number of classes).

- **Rows** → *actual* labels (ground truth)
- **Columns** → *predicted* labels (model output)
- **Diagonal** → correct predictions (*True Positives*)
- **Off-diagonal** → misclassifications

The Confusion Matrix helps identify which class pairs the model frequently confuses — information that cannot be obtained from the overall accuracy score alone.

```
!pip install seaborn
```

```
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.0.2)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2026.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
# Convert One Hot Encoded labels back to integers for the confusion matrix
y_true = np.argmax(
    y_test,
    axis=1
)

# Get ReLU model predictions on the entire test set
y_pred_relu = np.argmax(
    relu_model.predict(x_test),
    axis=1
)

cm_relu = confusion_matrix(
    y_true,
    y_pred_relu
)

plt.figure(figsize=(8,6))

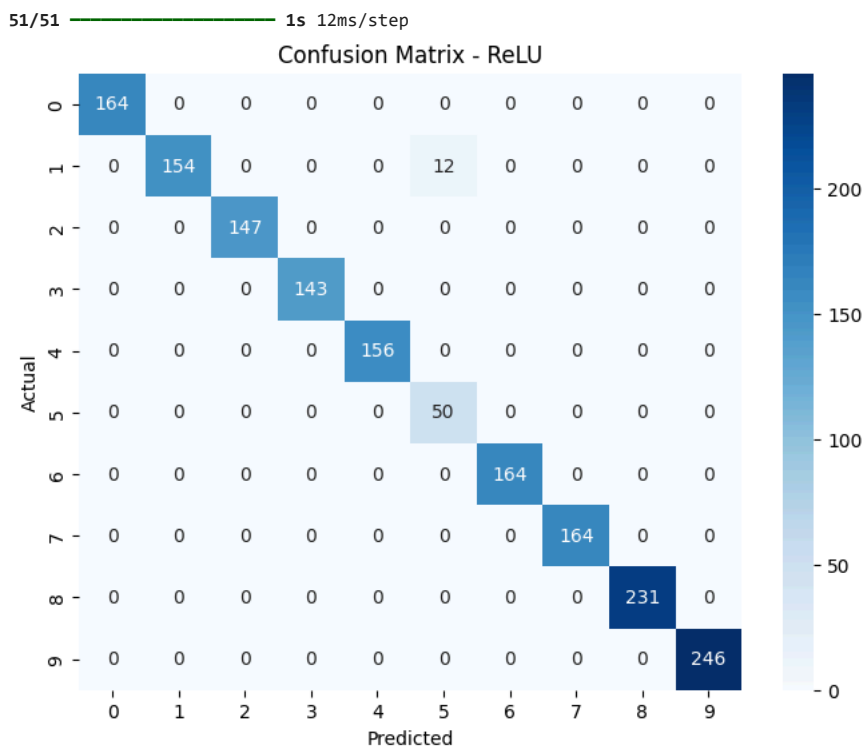
sns.heatmap(
    cm_relu,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title(
    'Confusion Matrix - ReLU'
)

plt.xlabel(
    'Predicted'
)

plt.ylabel(
    'Actual'
)

plt.show()
```



14. Confusion Matrix — Leaky ReLU

Confusion Matrix for the Leaky ReLU model. Compare this with the ReLU confusion matrix above to observe differences in misclassification patterns between the two models.

```
# Get Leaky ReLU model predictions on the entire test set
y_pred_leaky = np.argmax(
    leaky_model.predict(x_test),
    axis=1
)

cm_leaky = confusion_matrix(
    y_true,
    y_pred_leaky
)

plt.figure(figsize=(8,6))

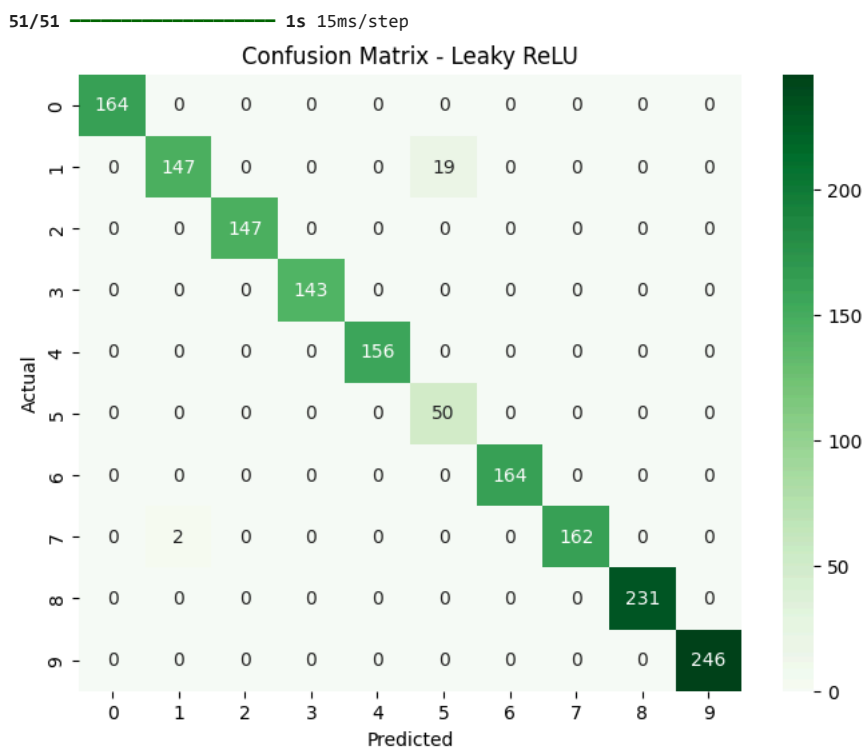
sns.heatmap(
    cm_leaky,
    annot=True,
    fmt='d',
    cmap='Greens'
)

plt.title(
    'Confusion Matrix - Leaky ReLU'
)

plt.xlabel(
    'Predicted'
)

plt.ylabel(
    'Actual'
)

plt.show()
```



15. Test Single Prediction

Tests the predictions of both models on a **single random image** from the test data.

An image is randomly selected from `x_test`, and its actual label is compared against the predictions from both the ReLU and Leaky ReLU models. This provides a visual sanity check to confirm the model is classifying images correctly.

```
# Pick a random image from the test set
test_idx = np.random.randint(
    0,
    len(x_test)
)

test_img_flat = x_test[test_idx]

actual_label = np.argmax(
    y_test[test_idx]
```

```

)

# Display the image (reshape from vector back to 2D image)
plt.figure(figsize=(4,4))

plt.imshow(
    test_img_flat.reshape(
        IMG_SIZE,
        IMG_SIZE,
        3
    )
)

plt.title(
    f'Actual: {classes[actual_label]}'
)

plt.axis('off')

plt.show()

```

Actual: Strawberry 1



```

# Run predictions from both models on the same image
inp = np.expand_dims(
    test_img_flat,
    axis=0
)

relu_pred = relu_model.predict(
    inp,
    verbose=0
)

leaky_pred = leaky_model.predict(
    inp,
    verbose=0
)

relu_class = np.argmax(
    relu_pred
)

leaky_class = np.argmax(
    leaky_pred
)

print("="*50)
print(
    "TEST PREDICTION RESULT"
)
print("="*50)
print(
    f"Actual Label      : {classes[actual_label]}"
)
print(
    f"ReLU Prediction   : {classes[relu_class]}"
)
print(
    f"Leaky Prediction  : {classes[leaky_class]}"
)

```

```

=====
TEST PREDICTION RESULT
=====
Actual Label      : Strawberry 1
ReLU Prediction   : Strawberry 1
Leaky Prediction  : Strawberry 1

```


16. Prediction Comparison Table

Compares the predictions of both models on **20 random images** from the test set.

This table directly shows on which samples ReLU and Leaky ReLU agree or disagree, and whether either prediction matches the actual label.

```
print(
    "\n==== PREDICTION COMPARISON =====\n"
)

print(
    f"{'No':<5}"
    f"{'Actual':<20}"
    f"{'ReLU':<20}"
    f"{'Leaky':<20}"
)

print("-"*80)

# Select 20 random indices from the test set (without replacement)
sample_idx = np.random.choice(
    len(x_test),
    20,
    replace=False
)

for idx, i in enumerate(sample_idx):

    inp = np.expand_dims(
        x_test[i],
        axis=0
    )

    actual = np.argmax(
        y_test[i]
    )

    relu_pred = np.argmax(
        relu_model.predict(
            inp,
            verbose=0
        )
    )

    leaky_pred = np.argmax(
        leaky_model.predict(
            inp,
            verbose=0
        )
    )

    print(
        f"{idx+1:<5}"
        f"{classes[actual]:<20}"
        f"{classes[relu_pred]:<20}"
        f"{classes[leaky_pred]:<20}"
    )
```

==== PREDICTION COMPARISON =====			
No	Actual	ReLU	Leaky
1	Kiwi 1	Kiwi 1	Kiwi 1
2	Tomato 1	Tomato 1	Tomato 1
3	Avocado 1	Avocado 1	Avocado 1
4	Avocado 1	Avocado 1	Avocado 1
5	Papaya 1	Papaya 1	Papaya 1
6	Strawberry 1	Strawberry 1	Strawberry 1
7	Avocado 1	Avocado 1	Avocado 1
8	Tomato 1	Tomato 1	Tomato 1
9	Pear 10	Pear 10	Pear 10
10	Banana 1	Banana 1	Banana 1
11	Tomato 1	Tomato 1	Tomato 1
12	Tomato 1	Tomato 1	Tomato 1
13	Papaya 1	Papaya 1	Papaya 1
14	Tomato 1	Tomato 1	Tomato 1
15	Strawberry 1	Strawberry 1	Strawberry 1
16	Avocado 1	Avocado 1	Avocado 1
17	Tomato 1	Tomato 1	Tomato 1
18	Avocado 1	Avocado 1	Avocado 1

19	Kiwi 1	Kiwi 1	Kiwi 1
20	Avocado 1	Avocado 1	Avocado 1

17. Final Result & Save Best Model

Displays a summary of the final experiment results and determines the best model based on accuracy (with loss as a tiebreaker).

The best model's weights are saved to a `.weights.h5` file so they can be loaded by the Streamlit application without needing to retrain from scratch.

```
print("ReLU Accuracy :", relu_acc)
print("ReLU Loss      :", relu_loss)

print("Leaky Accuracy:", leaky_acc)
print("Leaky Loss     :", leaky_loss)
```

```
ReLU Accuracy : 0.9926425218582153
ReLU Loss      : 0.013818042352795601
Leaky Accuracy: 0.9871244430541992
Leaky Loss     : 0.030568648129701614
```

```
# Display a summary table and determine the final winner
print("="*50)
print(
    f"{'Model':<15}"
    f"{'Accuracy':>12}"
    f"{'Loss':>12}"
)
print("="*50)
print(
    f"{'ReLU':<15}"
    f"{'relu_acc:>12.4f}"
    f"{'relu_loss:>12.6f}"
)
print(
    f"{'Leaky ReLU':<15}"
    f"{'leaky_acc:>12.4f}"
    f"{'leaky_loss:>12.6f}"
)
print("="*50)

# Winner: higher accuracy wins
# If accuracy is tied, lower loss wins
if relu_acc > leaky_acc:
    winner = "ReLU"
elif leaky_acc > relu_acc:
    winner = "Leaky ReLU"
else:
    if relu_loss < leaky_loss:
        winner = "ReLU"
    else:
        winner = "Leaky ReLU"

print(f"\n🏆 WINNER : {winner}")
```

```
=====
Model           Accuracy      Loss
=====
ReLU             0.9926      0.013818
Leaky ReLU       0.9871      0.030569
=====
```

🏆 WINNER : ReLU